

CGGS Coursework 1 (Simulation): Rigid-Body Simulation

Kaila Chin

Computer Graphics: Geometry and Simulation (2024-2025)[SEM2]

Amir Vaxman

07/03/2025

Introduction

In this report, I will analyze the limitations and drawbacks of correct behavior in the CW1:Rigid Body Simulation. While the simulation accurately obeys physical laws, certain characteristics of the objects lend itself to inefficiencies and odd artifacts. Therefore, I will be examining the disadvantages of such behavior and why it happens.

Additionally, I will be discussing the performance of the implemented collision and constraint system along with its scalability in terms of broad-phase space subdivision and how it can improve efficiency.

Basic Rigid-Body Simulation (1.1 Free Movement and 1.2 Implementing Collisions)

In this section of the simulation, the goal was to integrate free movement and implement collisions. Having applied gravity, we observe the objects in free fall, moving through the ground. We then install collision handling and resolved interpenetration linearly and velocities by computing impulses through the penetration, contact normal, and CRCoeff to ensure no overlap between objects as they collide.

Key Observances and Issues

Although the simulation correctly applies impulses and outputs a technically accurate response, the simulation itself presents unrealistic physics as real world scenarios are not taken into account. Factors like elasticity, energy loss, and friction are disregarded. On the right is Figure 1 where it is shown how objects fall and bounce unrealistically as the row of diamond figures bounce and simply go straight back up. I believe this is because of the lack of friction that is taken into account in the simulation. The diamond-like-objects collide with the ground at such a small point of contact that the frictionless

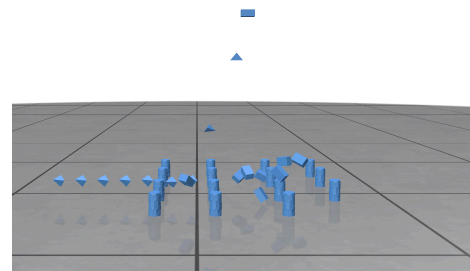


Figure 1: Two_chain.txt scene after it hits the ground

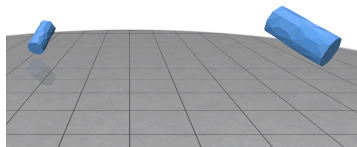


Figure 2: two_cylinder.txt scene as objects seem to float longer in the air

world simply causes them to bounce straight up. Similarly, while they respond quite differently, other objects seem to fly much higher than expected due to the frictionless arena and retention of energy. This could also be because we are using Euler integration to compute position and velocity, and because of its approximate calculations that don't preserve energy well, errors can occur over time. Thus, resulting in higher velocities than should be. Another disadvantage of correct behavior, as can be seen in Figure 2 and in the given video, is the lack of air resistance. Both cylinders bounce and glide through the air longer than normal with very little drag force, remaining in the air longer than usual as if by a lack of gravity. This differs from expectations as objects usually fall fast on the way down. Also, in this scene, the cylinders seem to either spin too fast or spin too slow, a side effect of the Euler integration again on rotation and velocities. Because of this, the simulation, although accurate,

presents a somewhat skewed image of physics as without friction and air resistance, objects that should realistically come to a stop or slide slowly instead bounce back with retained energy. As a result, in the future, implementing friction and air resistance could have more realistic outcomes

Extension: Working with Distance Constraints

This next section implements distance constraints by resolving positions and velocities. Through newly added holonomic constraints, position is resolved by using current distance between two points, reference value, and inverse masses. Velocity is then resolved through the use of a Jacobian matrix and Lagrange multiplier to calculate the impulse and update both linear and angular velocities.

Observations and Artifacts

We see with the constraints how the previous free objects are now constrained as they stay within the calculated constraints. The cube, previously several octahedrons, now form a bigger cube while the cylinders are now connected through two rubber-band constraints. Some artifacts notices from applying impulses to constraints can be seen in Figure 3. In the

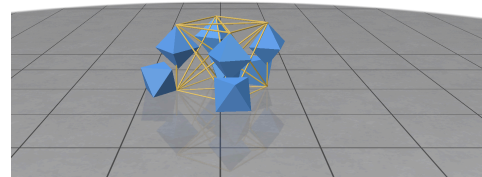


Figure 3: cube.txt scene jittering after hitting the ground

two
like

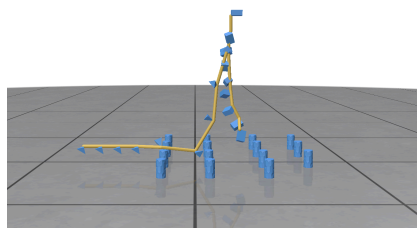


Figure 4: two_chain.txt scene

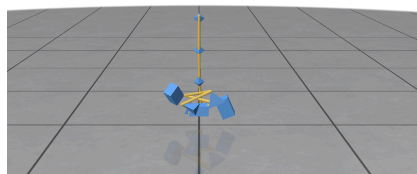


Figure 5: tower_chain.txt as it collides with a fixed object

cube.txt scene, the bounded cube falls to the ground, but instead of coming to a stop, the octahedrons continue to vibrate, spin, and sway in the air. This is also due to the lack of friction as each object continues to oscillate as they are smaller and more susceptible to movements. There is also energy conversation here as well where each object continues to move without resting long after the animation begins (as can be seen in the given video). This is most likely due to a combination of lack of friction and from using Euler integration again. Further, in Figure 4, the objects again react unrealistically as the chain perpendicular to the ground swings unnaturally to the left while, again, the diamond objects stay parallel to the ground. Looking at Figure 5, the chain unnaturally falls in a controlled heap, as the top objects begin to spin uncontrollably. But overall, these constraints provide a demonstration of a more controlled, realistic simulation.

Scalable and Efficient Rigid-Body Simulation

Broad-Phase Space Subdivision

For the last section of this report, we were tasked with implementing a broad-phase space subdivision that will allow

us to only check for the intersection of two meshes that are already close enough in space to better scale the simulation for a large quantity of objects. I decided to implement this through a spatial grid where I divided the space into cells and assigned objects to various cells (<https://developer.nvidia.com/gpugems/gpugems3/part-v-physics-simulation/chapter-32-broad-phase-collision-detection-cuda>). I created a new class Grid to do this by only checking the nearby objects in the same or neighboring cells, reducing the number of collision checks, which improved the overall efficiency and performance. I first cleared the spatial grid, before inserting objects into the grid, and once called, I used for loops to check if objects in cells were colliding and resolved collisions from that. However, when implementing this, I ran into some trouble with certain .txt files when running the simulation with “no-constraints.” And upon closer look, I realized that the problem was coming from the handling of fixed objects in collision detection and constraint resolution. I believe that when implementing my Grid.h class, the simulation would run out of bounds to access fixed points, and while I couldn’t find a solution to this error, the code still works for almost every scene besides “two_chain.txt.” I’ve added a video “two_chain.txt” running with constraints, and although the results are unrealistic, it outputs a somewhat normal response. On the left, I’ve included a photo of two_cylinder.txt running with my implemented spatial grid broad-phase space subdivision. Because of the errors running, I took the code out, but have added it on to the files, along with videos of it running.

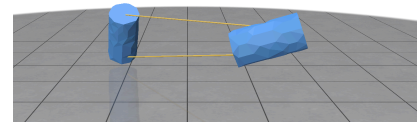


Image 6: two_cylinder.txt with broad-phase space subdivision

Efficient Constraint Scheduling

While I had trouble implementing an efficient constraint scheduling, I had tried implementing a flag to raise to check if that specific constraint would affect any others and if so, I’d check those next constraints and resolve them.

Conclusion

In this report, we explored the implementation of the rigid body simulation, discussed possible disadvantages to good behavior, and implemented extensions to make the program more efficient and smooth. Overall, while the code ran correctly, when analyzing its behavior, it sported unrealistic code because of lack of friction, air resistance, and computation errors. Through the implementation of broad-phase space subdivision, the program was enhanced and made more efficient by dividing the space into cells and only testing objects in the same cells or near the other for collisions.